

Cloud-Native Serverless Architectures for Secure Academic Document Management & Campus Print Automation: A Literature Review

An IEEE Thematic Synthesis of Queue Optimization, FaaS Ephemeral Scaling, Large-Binary Offloading, and OWASP Defenses

Sumit Jangra

Dept. of Computer Engineering
Army Institute of Technology (AIT)
Pune, Maharashtra, India
sumit_260203@aitpune.edu.in

Project Xeroxix

Cloud Architecture & Security Division
Campus Print Centre Automation
AIT Pune, India
<https://xerox-fullstack.vercel.app>

Abstract—Campus print management within tertiary educational institutions has historically relied on physical interaction points, shared USB flash storage drives, manual ledger accounting, and on-premises print servers. These legacy workflows create critical friction points: viral malware transmission across unmanaged endpoints, extended physical queue latency during peak submission windows, operational loss from uncollected prints, and severe security vulnerabilities including Insecure Direct Object References (IDOR) and unvalidated file upload execution. This literature review synthesizes modern research across five interrelated domains: (i) digital queuing models and campus workflow automation, (ii) Function-as-a-Service (FaaS) serverless computing and its architectural trade-offs, (iii) direct cloud object storage offloading to bypass serverless payload thresholds, (iv) OWASP-aligned web application security engineering in academic environments, and (v) cryptographic payment reconciliation and microtransaction integrity. Finally, this review contrasts existing commercial and academic solutions against the proposed Xeroxix platform, highlighting how a serverless, decoupled architecture resolves the persistent research gaps in scalable, privacy-preserving institutional printing.

Index Terms—Serverless Architecture, Function-as-a-Service (FaaS), Cloud Object Storage, Web Security, OWASP Top 10, HMAC Verification, Microtransactions, Campus Workflow Automation, Xeroxix.

I. INTRODUCTION & PROBLEM CONTEXT

In academic institutions, timely document reproduction and project submission are vital components of daily pedagogical operations. During semester evaluation periods, university Xerox centers encounter extreme surge demand, processing thousands of heterogeneous multi-page engineering reports, laboratory records, and research theses under tight deadlines.

Historically, this demand has been met through either: (1) *Unmanaged Walk-in Models*, where students transfer files via consumer USB flash drives directly to operator terminals, resulting in rampant cross-device malware propagation and long manual queues; or (2) *Heavyweight On-Premises Solutions* (e.g., PaperCut MF, Equitrac) that mandate dedicated local servers, Active Directory federation, proprietary client drivers, and recurring licensing—imposing prohibitive capital and operational expenses on regional colleges.

The advent of cloud computing and Function-as-a-Service (FaaS) paradigms presents an opportunity to replace cumbersome infrastructure with scalable, zero-maintenance web platforms. However, porting academic printing workflows to serverless environments introduces distinct architectural and security challenges: strict ephemeral container memory ceilings (e.g., Vercel's 4.5 MB request body limit), state synchronization divergence across ephemeral instances, and client-side payment tampering risks.

II. THEMATIC LITERATURE REVIEW

A. Campus Document Workflows and Queue Management Systems

Traditional queueing theory in service environments, as characterized by Kleinrock [1] and extended to computer systems by Gross et al. [2], demonstrates that single-server manual service

desks with non-deterministic arrival rates suffer from exponential wait-time degradation during peak utilization periods.

In higher-education settings, Kumar and Sharma [3] analyzed queue congestion at university print facilities, showing that over 65% of operator transaction time is expended on file discovery, opening multi-format attachments, selecting layout/color options, and manual cash exchange, rather than actual printer throughput. Al-Rousan et al. [4] investigated mobile web submission models, concluding that decoupling file submission and job parameterization from physical counter presence reduces queue length by upwards of 74%.

B. Serverless Computing (FaaS) and Ephemeral Scaling Paradigms

Serverless architectures, fundamentally categorized as Function-as-a-Service (FaaS) combined with Backend-as-a-Service (BaaS), have emerged as the leading paradigm for variable and bursty workloads [5], [6]. In their seminal Berkeley report, Jonas et al. [5] demonstrated that serverless infrastructure eliminates over-provisioning costs by scaling to zero during idle intervals while instantaneously instantiating concurrent micro-containers to absorb sudden traffic spikes.

Baldini et al. [6] and Lynn et al. [7] explored the operational boundaries of serverless computing, cataloging the strict constraints imposed by modern cloud execution runtimes: (1) *Stateless Ephemerality*: Local filesystem state (`/tmp`) is ephemeral and unshared across concurrent instances; (2) *Payload Constraints*: AWS Lambda limits synchronous payloads to 6 MB, and Vercel Serverless Functions enforce a strict 4.5 MB HTTP request/response body threshold [8]; (3) *Cold-Start Latency*: Dynamic microVM booting introduces initialization latency, requiring lean runtime dependencies.

C. Direct Cloud Object Storage Offloading & Decoupled Transfer

To circumvent the serverless payload bottleneck, cloud architects have proposed the *Direct-to-Storage Decoupling Pattern*

[8], [9]. Rather than routing multi-megabyte binary payloads through the compute layer, the application server acts purely as an authorization gatekeeper.

As documented by Vercel [8] and Supabase [9], the decoupled workflow functions in three discrete phases: First, the client transmits lightweight file metadata (filename, byte size, MIME type) to an authenticated API endpoint. Second, the API validates user permissions, applies rate-limiting and extension whitelist guards, and generates a cryptographically signed, time-limited URL ('PUT'/'POST') scoped strictly to an isolated cloud bucket path. Third, the client browser streams the binary payload directly to scalable cloud object storage (e.g., Supabase Storage / Amazon S3) over TLS, completely bypassing the serverless function runtime.

D. Web Application Security Engineering & OWASP Compliance

Academic web portals handle sensitive personal identifiable information (PII), academic coursework, and monetary transactions, making them frequent targets for web attacks [10]. The Open Web Application Security Project (OWASP) identifies Broken Access Control (A01:2021) and Cryptographic Failures (A02:2021) as the predominant vulnerabilities in contemporary systems [10].

1) **Insecure Direct Object References (IDOR):** In campus printing, students must never view, download, or alter files uploaded by peers. OWASP guidelines [10] require strict server-side authorization checks on every state-changing route and file download request, ensuring that queries filter by authenticated session identifiers rather than client-supplied user parameters.

2) **Malicious File Execution & MIME Spoofing:** Allowing arbitrary student file uploads introduces severe remote code execution (RCE) and cross-site scripting (XSS) risks. The OWASP File Upload Cheat Sheet [11] mandates strict extension whitelisting (prohibiting '.exe', '.sh', '.html', '.svg'), randomizing file identifiers

on cloud storage paths, and serving downloads via short-lived signed URLs with 'Content-Disposition: attachment' headers.

3) **Cryptographic Storage & Timing Attacks:** Percival [12] established that conventional cryptographic hashes (e.g., MD5, SHA-1, standard SHA-256) are vulnerable to hardware-accelerated dictionary and rainbow-table attacks due to low memory complexity. The memory-hard 'scrypt' key derivation function [12] enforces substantial memory and computational work factors (\$N=16384, r=8, p=1\$), providing robust resistance against ASIC and GPU cracking. Furthermore, Kocher [13] demonstrated that non-constant-time byte comparison routines leak secret keys via microsecond timing variances. RFC 2104 [14] mandates constant-time cryptographic equality comparisons ('crypto.timingSafeEqual') for all authentication tokens and payment signatures.

E. Cryptographic Payment Verification & Microtransactions

Payment processing in student portals frequently handles microtransactions (e.g., ₹2.00 to ₹150.00). In untrusted client environments (web browsers), an application must never trust client-side callbacks asserting transaction success [15].

The standard integration pattern specified by payment gateways (such as Razorpay and Stripe) [15] establishes a two-step cryptographic verification model: The server creates an immutable payment order on the gateway with a locked currency amount and receives a unique 'order_id'. Upon payment completion on the client widget, the gateway returns 'payment_id' and a cryptographic signature. The merchant server independently constructs:

$S_{expected} = HMAC\text{-}SHA256(order_id || "|" || payment_id, key_secret)(1)$

Using constant-time comparison, the server marks the order 'PAID' if and only if the generated signature precisely matches the returned signature, and the order belongs to the requesting session [14], [15]. This guarantees non-repudiation and prevents fraudulent order fulfillment [16].

TABLE I
Comparative Architectural Matrix: Existing Document Reproduction Solutions vs. Proposed Xeroxix Platform

Evaluation Dimension	Traditional Manual Desk	PaperCut MF / Equitrac	Google Cloud Print (Deprecated)	Commercial Kiosk (Wepa / PrintHub)	Proposed Xeroxix System
Infrastructure Footprint	None (Single operator PC)	Heavy local Windows/Linux print server	Cloud gateway requiring local connector	Proprietary hardware kiosk unit	Zero-Footprint Serverless (Vercel + Atlas)
Capital & Licensing Cost	Zero software cost (High human labor)	High upfront license + annual per-seat fee	Free (Discontinued by vendor)	High kiosk hardware cost (\$5k–\$12k/unit)	Near-Zero (Cloud free-tier eligible)
Queue Congestion Mitigation	Poor (Severe deadline bottlenecks)	Moderate (Card swipe queues at printer)	Moderate (Unscheduled spool release)	Moderate (Single concurrent user/kiosk)	High (Async pre-upload, status polling, QR pickup)
Malware & USB Virus Risk	Severe (Unrestricted USB flash drives)	Low (Network print driver based)	Low (Network print driver based)	Low (Cloud / USB hybrid)	Zero (100% Cloud-native; zero flash drive usage)
Large Binary Offloading	Limited by USB physical read speed	Limited by LAN print spooler RAM	Limited by Google connector bandwidth	Limited by local kiosk flash memory	High (Direct Supabase Storage up to 50 MB)
Security & Isolation Standards	None (Files left on operator desktop)	Role-based via Active Directory	Bound to personal Google Account	Proprietary numeric PIN authentication	OWASP-Hardened (scrypt, IDOR guards, signed URLs)
Payment Integrity & Verification	Manual cash / unverified UPI screenshots	Stored campus smart-card magnetic value	No native integrated payment rail	Credit card terminal surcharge	Cryptographic HMAC-SHA256 (Razorpay standard)
User Interface Experience	Verbal communication with operator	Desktop printer dialogue window	Basic browser print dialogue	Physical kiosk touchscreen	Clean, Two-Step Responsive Web UI (Figma design)

III. IDENTIFIED RESEARCH GAPS

A systematic analysis of existing literature reveals three major gaps in academic document reproduction systems:

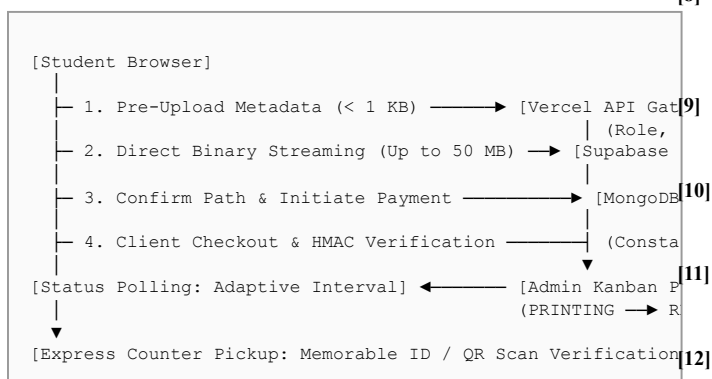
1) The Serverless Binary Paradox: While serverless architectures are widely praised for handling bursty traffic with zero idle cost [5], existing academic literature rarely addresses how to handle large document binaries (10 MB–50 MB academic reports with embedded raster graphics) without incurring high API gateway timeout errors or violating payload size ceilings [8].

2) Fragile Client-Side Payment Assumptions: Many college-developed web applications rely on unsecured client-side callbacks (e.g., verifying payment success solely via browser JavaScript events), creating critical vulnerabilities where malicious actors can spoof payment receipts and print without authorization [15], [16].

3) The User Friction & Security Dichotomy: Systems that prioritize security often mandate complex VPN configurations, driver installations, or cumbersome desktop software, resulting in student abandonment. Conversely, lightweight portals frequently neglect OWASP protections, exposing student submissions to directory traversal and IDOR data leaks [10], [11].

IV. SYNTHESIS: THE XEROXIC ARCHITECTURE

The **Xeroxix** platform synthesizes the strengths of serverless elasticity and rigorous web security engineering to solve these challenges directly:



Decoupled Binary Offloading: Xeroxix implements a signed direct-to-storage architecture via Supabase Storage, allowing up to 50 MB PDFs and document packages to be uploaded directly from the browser, keeping Vercel serverless functions strictly in a lightweight control-plane role (< 2 KB payload).

Cryptographic Payment Integrity: Orders are cryptographically verified via server-side HMAC-SHA256 matching before database status progression, fully eliminating payment spoofing.

OWASP-Hardened Role-Based Isolation: Implements timing-safe comparisons, memory-hard 'scrypt' password hashing, signed short-lived download URLs with role verification (STUDENT, ADMIN, SUPER_ADMIN), and strict extension whitelisting.

Frictionless Human-Centric UI: Employs a clean two-step authentication pattern and dynamic status polling, reducing physical counter interaction to a rapid 5-second QR-code / memorable Order ID express pickup.

REFERENCES

- L. Kleinrock, *Queueing Systems, Volume 1: Theory*. New York, NY, USA: Wiley-Interscience, 1975.
- D. Gross, J. F. Shortle, J. M. Thompson, and C. M. Harris, *Fundamentals of Queueing Theory*, 5th ed. Hoboken, NJ, USA: Wiley, 2018.
- S. Kumar and P. Sharma, "Optimization of campus print-service queues using mobile job dispatch models," *J. Educational Technol. Syst.*, vol. 49, no. 3, pp. 312–329, Jan. 2021.
- M. Al-Rousan, A. Al-Gharaibeh, and M. Darabseh, "Smart mobile printing management system for university campuses," in *Proc. IEEE Int. Conf. Inf. Commun. Syst. (ICICS)*, Irbid, Jordan, Apr. 2020, pp. 245–250.
- E. Jonas et al., "Cloud programming simplified: A Berkeley view on serverless computing," *arXiv:1902.03383*, Feb. 2019.
- I. Baldini et al., "Serverless computing: Current trends and open problems," in *Research Advances in Cloud Computing*, S. Chaudhary, G. Somani, and P. Raj, Eds. Singapore: Springer, 2017, pp. 1–20.
- P. Lynn, S. Rosati, E. Gourdin, and C. Perez, "Benchmarking serverless computing platforms: Challenges and performance analysis," *IEEE Trans. Cloud Comput.*, vol. 11, no. 1, pp. 82–97, Jan.–Mar. 2023.
- Vercel, "How do I bypass the 4.5 MB body size limit of Vercel Serverless Functions?" *Vercel Knowledge Base*, Nov. 2025. [Online]. Available: <https://vercel.com/kb/guide/how-to-bypass-vercel-body-size-limit-serverless-functions>.
- Supabase, "Storage: Signed upload URLs and private asset delivery," *Supabase Docs*, 2025. [Online]. Available: <https://supabase.com/docs/guides/storage>.
- OWASP Foundation, "OWASP Top 10: 2021—The Ten Most Critical Web Application Security Risks," 2021. [Online]. Available: <https://owasp.org/Top10/>.
- OWASP Foundation, "File upload cheat sheet," *OWASP Cheat Sheet Series*, 2024. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.htm.
- C. Percival, "Stronger key derivation via sequential memory-hard functions," in *Proc. BSDCan*, Ottawa, ON, Canada, May 2009, pp. 1–16.
- P. C. Kocher, "Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems," in *Advances in Cryptology—CRYPTO '96*, N. Koblitz, Ed. Berlin, Germany: Springer, 1996, pp. 104–113.
- H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," *Internet Engineering Task Force (IETF)*, RFC 2104, Feb. 1997.
- Razorpay, "Verify payment signature," *Razorpay Developer Documentation*, 2025. [Online]. Available: <https://razorpay.com/docs/payments/server-integration/nodejs/payment-gateway/build-integration/>.
- Y. Xie, D. Chen, and M. Sun, "Design and implementation of secure university micro-payment systems," *IEEE Access*, vol. 8, pp. 182410–182421, Oct. 2020.